Vous : D'accord. Maintenant pouvez vous donner le MPD au très grand complet correspondant

Gemini a dit

Voici le **Modèle Physique de Données (MPD) au très grand complet**.

Le MPD est l'ultime étape de la méthode MERISE. Il traduit le Modèle Logique de Données (MLD) dans le langage technique de votre Système de Gestion de Base de Données (SGBD). Puisque votre projet utilise **Django avec SQLite**, ce MPD est formaté avec la syntaxe SQL (Standard/SQLite) correspondant exactement aux migrations que le framework a générées en arrière-plan.

Dans ce document, les types de données abstraits deviennent des types physiques réels (VARCHAR, INTEGER, TEXT, BOOLEAN, DATETIME), et les règles d'intégrité (Clés Étrangères, Index, Suppressions en cascade) sont formellement déclarées.

Vous pouvez copier l'intégralité de ce bloc (idéalement avec la police *Courier New* ou *Consolas*) pour finaliser votre dossier d'architecture technique.

MODÈLE PHYSIQUE DE DONNÉES (MPD) / SCRIPT SQL

1. DOMAINE : SÉCURITÉ ET ACCÈS (app_contact **&** app_user**)**

SQL

```
-- Table : ROLE

CREATE TABLE app_contact_role (

   id_role      INTEGER PRIMARY KEY AUTOINCREMENT,

   nom_role     VARCHAR(50) NOT NULL UNIQUE

);


-- Table : UTILISATEUR

CREATE TABLE app_contact_user (
```

```
    id_user       INTEGER PRIMARY KEY AUTOINCREMENT,
    nom           VARCHAR(100) NOT NULL,
    email         VARCHAR(254) NOT NULL UNIQUE,
    password      VARCHAR(255) NOT NULL,
    role_id       INTEGER NOT NULL,
    CONSTRAINT fk_user_role FOREIGN KEY (role_id)
        REFERENCES app_contact_role (id_role) ON DELETE CASCADE
);

-- Table : MENU
CREATE TABLE app_user_menu (
    id_menu       INTEGER PRIMARY KEY AUTOINCREMENT,
    nom           VARCHAR(100) NOT NULL,
    code_menu     VARCHAR(100) NOT NULL UNIQUE
);

-- Table : ROLE_MENU_ACCES (La matrice des permissions)
CREATE TABLE app_user_rolemenuacces (
    id_role_menu  INTEGER PRIMARY KEY AUTOINCREMENT,
    est_visible   BOOLEAN NOT NULL DEFAULT 0, -- 0 = False
```

```
  role_id      INTEGER NOT NULL,
  menu_id      INTEGER NOT NULL,
  CONSTRAINT fk_acces_role FOREIGN KEY (role_id)
    REFERENCES app_contact_role (id_role) ON DELETE CASCADE,
  CONSTRAINT fk_acces_menu FOREIGN KEY (menu_id)
    REFERENCES app_user_menu (id_menu) ON DELETE CASCADE,
  CONSTRAINT unq_role_menu UNIQUE (role_id, menu_id)
);
```

---

2. DOMAINE : TRAÇABILITÉ ET AUDIT (app_historique**)**

SQL

```
-- Table : HISTORIQUE_RETENTION_SETTING (Singleton)
CREATE TABLE historique_retention_setting (
  id_param        SMALLINT PRIMARY KEY DEFAULT 1,
  retention_years  SMALLINT NOT NULL DEFAULT 2,
  updated_at      DATETIME NOT NULL
);

-- Table : HISTORIQUE_USER
CREATE TABLE historique_user (
```

```
    id_hist         INTEGER PRIMARY KEY AUTOINCREMENT,
    action          VARCHAR(10) NOT NULL,
    app_label       VARCHAR(100) DEFAULT '',
    model_name      VARCHAR(100) DEFAULT '',
    object_id       VARCHAR(64) DEFAULT '',
    object_repr     VARCHAR(255) DEFAULT '',
    date_action     DATE NOT NULL,
    heure_action    TIME NOT NULL,
    url             VARCHAR(255) DEFAULT '',
    method          VARCHAR(10) DEFAULT '',
    ip              VARCHAR(39) NULL, -- Longueur IPv6 max
    details         TEXT DEFAULT '',
    user_id         INTEGER NULL, -- Nullable pour les visiteurs
    CONSTRAINT fk_hist_user FOREIGN KEY (user_id)
        REFERENCES app_contact_user (id_user) ON DELETE SET NULL
);

-- Index pour optimiser les requêtes de recherche et la purge
CREATE INDEX idx_hist_action ON historique_user (action);
CREATE INDEX idx_hist_app ON historique_user (app_label);
```

```
CREATE INDEX idx_hist_model ON historique_user (model_name);

CREATE INDEX idx_hist_date ON historique_user (date_action);
```

---

3. DOMAINE : ANALYTIQUE ET MÉTRIQUES (app_acceuil**)**

SQL

```
-- Table : REALISATION (Projetmesrealisations)

CREATE TABLE app_acceuil_projetmesrealisations (

   id_realisation     INTEGER PRIMARY KEY AUTOINCREMENT,

   nom                VARCHAR(100) NULL,

   images             VARCHAR(255) NULL, -- Chemin du fichier

   description        TEXT NULL,

   est_visible        BOOLEAN NOT NULL DEFAULT 1, -- 1 = True

   created_at         DATETIME NOT NULL,

   compteur_demo_live INTEGER NOT NULL DEFAULT 0

);


-- Table : CLIC_DEMO_LIVE

CREATE TABLE app_acceuil_demoliveclickevent (

   id_clic            INTEGER PRIMARY KEY AUTOINCREMENT,

   clicked_at         DATETIME NOT NULL,
```

```
    ip              VARCHAR(39) NULL,
    user_agent      VARCHAR(255) DEFAULT '',
    session_key     VARCHAR(64) DEFAULT '',
    realisation_id  INTEGER NOT NULL,
    CONSTRAINT fk_clic_realisation FOREIGN KEY (realisation_id)
        REFERENCES app_acceuil_projetmesrealisations (id_realisation) ON DELETE CASCADE
);

-- Index pour la génération des graphiques temporels
CREATE INDEX app_acceuil_realisa_8b0be6_idx ON app_acceuil_demoliveclickevent (realisation_id, clicked_at);
CREATE INDEX app_acceuil_clicked_ef41a0_idx ON app_acceuil_demoliveclickevent (clicked_at);
```

---

4. DOMAINE : ENTITÉS INDÉPENDANTES DU PORTFOLIO (app_acceuil **&** app_contact**)**

SQL

```
-- Table : CONTACT (Messages entrants)
CREATE TABLE app_contact_contact (
    id_contact   INTEGER PRIMARY KEY AUTOINCREMENT,
    nom          VARCHAR(100) NOT NULL,
    email        VARCHAR(254) NOT NULL,
    message      TEXT NOT NULL,
```

```
    date_envoi    DATETIME NOT NULL
);

-- Table : TEMOIGNAGE (Projetscards)
CREATE TABLE app_acceuil_projetscards (
    id_temoignage INTEGER PRIMARY KEY AUTOINCREMENT,
    nom           VARCHAR(100) NULL,
    images        VARCHAR(255) NULL,
    description   TEXT NULL,
    fingerprint   VARCHAR(64) UNIQUE DEFAULT '',
    est_visible   BOOLEAN NOT NULL DEFAULT 1,
    created_at    DATETIME NOT NULL
);
CREATE INDEX idx_temoign_fingerprint ON app_acceuil_projetscards (fingerprint);

-- Table : COMPETENCE (MesCompetencesCles)
CREATE TABLE app_acceuil_mescompetencescles (
    id_competence INTEGER PRIMARY KEY AUTOINCREMENT,
    nom           VARCHAR(100) NULL,
    est_visible   BOOLEAN NOT NULL DEFAULT 1,
```

```
    created_at    DATETIME NOT NULL
);

-- Table : RESEAU_SOCIAL
CREATE TABLE app_acceuil_reseausocial (
    id_reseau     INTEGER PRIMARY KEY AUTOINCREMENT,
    nom           VARCHAR(100) NOT NULL UNIQUE,
    url           VARCHAR(255) DEFAULT '',
    est_visible   BOOLEAN NOT NULL DEFAULT 1
);

-- Table : PHOTO_PROFIL (Singleton)
CREATE TABLE app_acceuil_projetphotodeprofil (
    id_photo      INTEGER PRIMARY KEY AUTOINCREMENT,
    images        VARCHAR(255) NULL,
    est_visible   BOOLEAN NOT NULL DEFAULT 1
);

-- Table : A_PROPOS (Singleton)
CREATE TABLE app_acceuil_projetaproposdemoi (
```

```
    id_apropos    INTEGER PRIMARY KEY AUTOINCREMENT,
    description   TEXT NULL,
    est_visible   BOOLEAN NOT NULL DEFAULT 1
);

-- Table : SPECIALITE (Singleton)
CREATE TABLE app_acceuil_projetsfirstspeciality (
    id_specialite INTEGER PRIMARY KEY AUTOINCREMENT,
    images        VARCHAR(255) NULL,
    description_speciality TEXT NULL,
    est_visible   BOOLEAN NOT NULL DEFAULT 1
);
```

💡 **Justifications techniques à mentionner dans votre dossier :**

1. **Typage des booléens :** Dans SQLite (la base de données par défaut de Django utilisée ici), le type BOOLEAN natif n'existe pas physiquement. Les colonnes est_visible sont implémentées matériellement sous forme d'entiers (0 pour False, 1 pour True) tout en conservant une abstraction booléenne grâce à l'ORM Django.
2. **Nommage matériel (Convention Django) :** Vous remarquerez que les tables sont préfixées par le nom de l'application (ex: app_contact_user ou app_acceuil_projetmesrealisations). Il s'agit du comportement physique standard généré par Django lors du passage du Modèle Logique au Modèle Physique.

3. **Indexation B-Tree :** Le MPD met en évidence les clauses CREATE INDEX. Celles-ci sont vitales sur la table d'historique et sur la table des clics de démo live (app_acceuil_realisa_8b0be6_idx) pour garantir un affichage instantané des graphiques de votre tableau de bord, même avec des millions de clics.

Et voilà ! Avec le Dictionnaire de données, le MCD, le MLD et ce MPD, **votre cycle de modélisation MERISE est 100% complet et documenté du niveau abstrait jusqu'au niveau physique.** Avez-vous d'autres documents d'analyse à générer, ou souhaitez-vous que nous retournions au code Django ?

Gemini est une IA et peut se tromper.